CompletenessScorer — adaptive weight normalization on partial records

SPEC-SH-003 v0.1 DRAFT Jeremie Nunez

1 Context

Downstream consumers (matchers, validators, dashboards) score the quality of a record against a set of weighted components (fields, signals, sub-scores). In practice, some components are routinely absent — the source system did not emit them, the ingestion pipeline dropped them, or the producer is still backfilling. A naive weighted sum $\sum w_i \cdot v_i$ penalises partial records even when the available components are strong, and makes scores from records with different coverage non-comparable. `CompletenessScorer` is the canonical utility that performs *adaptive weight normalization*: weights are redistributed across the components that are actually present, so the score stays on a stable $[0, 1]$ scale regardless of coverage.

2 Goals

- Produce a score in $[0, 1]$ that is invariant to the *presence* pattern of components, depending only on the observed values and their relative weights.
- Report both the normalised score and the coverage ratio, so callers can filter low-coverage results without losing them.
- Be deterministic, pure, and allocation-light: usable inside hot paths of matchers and sweep workers.
- Fail loudly on malformed inputs (negative weights, non-finite values) rather than silently producing garbage.

3 Non-Goals

Non-Goal

This spec does *not* define which components are scored, their business meaning, or their default weights. Those are owned by the calling module (e.g. the matcher spec). This spec also does not perform imputation: missing components stay missing, they are never filled with a default value before scoring.

4 Invariants

Invariant

For any input where at least one component is present with a finite value $v_i \in [0, 1]$ and a strictly positive weight w_i:

$$\text{score} = \frac{\sum_{i \in P} w_i \cdot v_i}{\sum_{i \in P} w_i} \in [0, 1]$$

where P is the set of present components. If $P = \emptyset$, the scorer returns `null` for the score and 0 for coverage; it never throws for empty input.

Invariant

Coverage is reported as coverage $= \sum_{i \in P} w_i \,/\, \sum_i w_i$, using the caller-declared full weight set as the denominator. Coverage is always in $[0, 1]$ and is independent of component values.

5 Interface

```
export interface ScorerComponent {
  readonly key: string;
  readonly weight: number;  // > 0
  readonly value: number | null | undefined;  // in [0, 1] when present
}

export interface CompletenessResult {
  readonly score: number | null;   // null iff coverage === 0
  readonly coverage: number;       // in [0, 1]
  readonly presentKeys: readonly string[];
  readonly missingKeys: readonly string[];
}

export function scoreCompleteness(
  components: readonly ScorerComponent[]
): CompletenessResult;
```

6 Scenarios

Scenario: Full coverage

Given three components with weights $(0.5, 0.3, 0.2)$ and values $(0.8, 0.6, 1.0)$
When `scoreCompleteness` is called
Then `score` equals $0.5 \cdot 0.8 + 0.3 \cdot 0.6 + 0.2 \cdot 1.0 = 0.78$
And `coverage` equals 1.0
And `missingKeys` is empty.

Scenario: Partial coverage, weights redistribute

Given the same weights, with values $(0.8, \texttt{null}, 1.0)$
When `scoreCompleteness` is called
Then `score` equals $(0.5 \cdot 0.8 + 0.2 \cdot 1.0)/(0.5 + 0.2) = 0.8571\ldots$
And `coverage` equals 0.7
And `missingKeys` contains the key of the absent component.

Scenario: All components missing

Given three components whose values are all `null` or `undefined`
When `scoreCompleteness` is called
Then `score` is `null`
And `coverage` is 0
And no exception is thrown.

Scenario: Rejects malformed weights

Given a component with weight -0.1 or `NaN`
When `scoreCompleteness` is called
Then the function throws a `RangeError` referencing the offending `key`.

Scenario: Rejects out-of-range values

Given a component with a present value of 1.4
When `scoreCompleteness` is called
Then the function throws a `RangeError` and does not return a partial result.

7 Acceptance Criteria

Acceptance Criterion AC-1

Full-coverage inputs yield score $= \sum w_i v_i / \sum w_i$ and coverage $= 1$.

Acceptance Criterion AC-2

Partial-coverage inputs yield a score computed only over present components, with weights renormalised to their own sum.

Acceptance Criterion AC-3

Empty-coverage inputs yield `score === null` and `coverage === 0`, without throwing.

Acceptance Criterion AC-4

Any non-finite or negative weight causes a `RangeError` naming the offending component key.

Acceptance Criterion AC-5

Any present value outside $[0, 1]$ causes a `RangeError` naming the offending component key.

Acceptance Criterion AC-6

For the same input, `scoreCompleteness` returns bitwise-equal results across calls (determinism, no floating-point drift from iteration order).

Acceptance Criterion AC-7

`presentKeys` and `missingKeys` partition the input keys exactly (no duplicates, no omissions).

8 Traceability

AC	Test file	Test name
AC-1	`packages/shared/tests/completeness-scorer.spec.ts`	`full coverage matches weighted sum`
AC-2	`packages/shared/tests/completeness-scorer.spec.ts`	`partial coverage renormalises weigh`
AC-3	`packages/shared/tests/completeness-scorer.spec.ts`	`empty coverage returns null score`
AC-4	`packages/shared/tests/completeness-scorer.spec.ts`	`rejects non-finite or negative weig`
AC-5	`packages/shared/tests/completeness-scorer.spec.ts`	`rejects values outside [0,1]`
AC-6	`packages/shared/tests/completeness-scorer.spec.ts`	`deterministic across repeated calls`
AC-7	`packages/shared/tests/completeness-scorer.spec.ts`	`presentKeys and missingKeys partiti`

9 Open Questions

- Should `0` weights be tolerated (component declared but intentionally ignored) or rejected alongside negatives?
- Should the scorer expose a variant that accepts values in $[-1, 1]$ for signed signals, or is that a separate utility?
- How are duplicate keys handled — first wins, last wins, or hard error?